AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

...Algorytm listy dwukierunkowej z zastosowaniem GitHub...

Autor:
Kacper Twaróg

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2024

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **4**

3. Projektowanie **6**

4. Implementacja **7**

5. Wnioski **9**

Literatura **10**

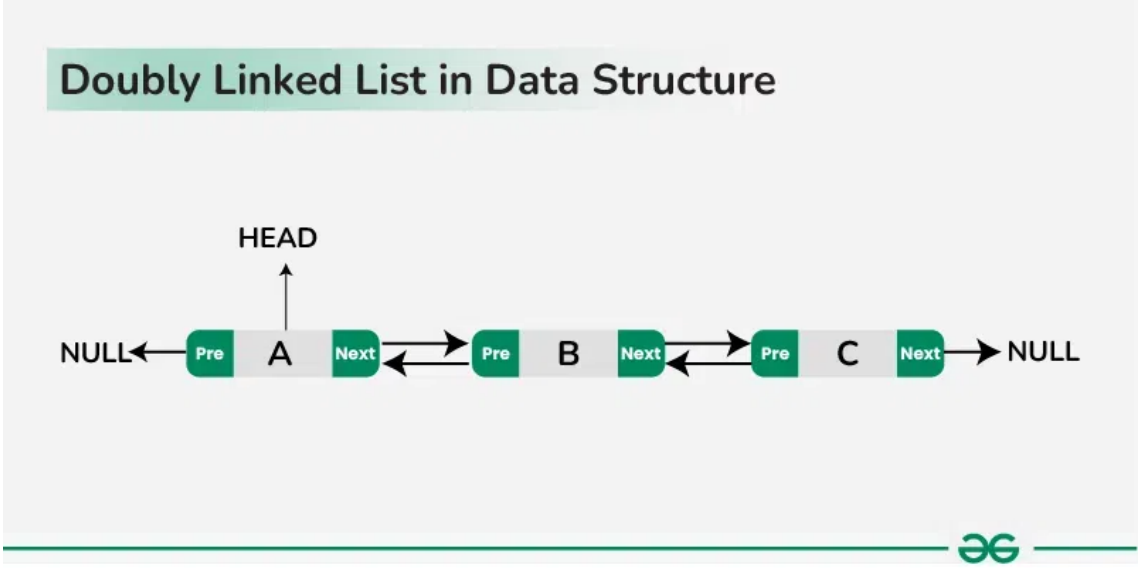
Spis rysunków **10**

Spis tabel **11**

Spis listingów **12**

1. Ogólne określenie wymagań

Ten projekt w C++ polega na tworzeniu listy dwukierunkowej, a także na wykorzystaniu metod i pomysłów, takich jak metody fabrykujące czy koncepcja iteracji/iteratorów. To również sposób na ćwiczenie wykonywania commitów w GitHubie i zapoznanie się z przebiegiem pracy.



Rys. 1.1. Przykład jak lista działa

Cała konfiguracja korzysta z GitHuba do zarządzania projektem. Zamiast Gita używany jest GitHub Desktop, aby mieć graficzny interfejs do scalania, pobierania i wysyłania zmian oraz innych funkcji GitHuba.

Implementacja wymaga trzech głównych komponentów: metody fabrykującej do tworzenia, iteracji umożliwiającej przechodzenie przez całą sekwencję węzłów oraz implementacji listy dwukierunkowej. Należy je umieścić w osobnych plikach .cpp i .h. Plik main.cpp testuje funkcje za pomocą różnych operacji.

2. Analiza problemu

Doubly Linked List

Jest to liniowa struktura danych, w której każdy węzeł zawiera dane oraz dwa wskaźniki łączące go z poprzednim i następnym węzłem w sekwencji. To dwukierunkowe połączenie umożliwia bardzo wydajne wstawianie i usuwanie elementów w dowolnym miejscu bez przesuwania pozostałych elementów w pamięci, w przeciwieństwie do standardowych tablic. Rozwiązuje problem sztywnego przydzielania pamięci, umożliwiając dynamiczne zwiększanie i zmniejszanie niestandardowych struktur oraz płynne przechodzenie po nich w obu kierunkach.

Iteration

Iteracja zapewnia ustandaryzowany mechanizm przechodzenia przez elementy kolekcji, na przykład po kolei przez węzły oparte na wskaźnikach w niestandardowej liście. Hermetyzuje logikę przechodzenia, co oznacza, że można iterować po złożonych strukturach bez ujawniania reszcie aplikacji szczegółów zarządzania ich pamięcią wewnętrzną. Rozwiązuje to problem ścisłego powiązania, zapewniając, że standardowe algorytmy mogą bezpiecznie odczytywać niestandardowe obiekty i wchodzić z nimi w interakcję, niezależnie od tego, jak zorganizowana jest ich pamięć.

Factory Method

Ten kreacyjny wzorzec projektowy definiuje interfejs do tworzenia obiektu, ale deleguje dokładną logikę przydzielania pamięci na podklasy lub do konkretnych metod. Podczas tworzenia złożonych hierarchii obiektów w C++ zapobiega zakodowaniu systemu na stałe pod kątem konkretnych klas, dzięki czemu architektura pozostaje elastyczna i modułowa. Rozwiązuje problem kruchego kodu, centralizując tworzenie obiektów, co znacznie ułatwia wprowadzanie nowych typów obiektów bez przepisywania istniejącej logiki klienta, która z nich korzysta.

Github

GitHub to oparta na chmurze platforma hostingowa wykorzystująca system kontroli wersji Git, która umożliwia programistom przechowywanie kodu źródłowego, zarządzanie nim i śledzenie wprowadzanych w nim zmian na przestrzeni czasu. Zapewnia centralne miejsce do tworzenia kopii zapasowych kodu, przechowywania zintegro-

wanej dokumentacji i płynnej współpracy w różnych środowiskach programistycznych. Rozwiązuje problem chaotycznego procesu tworzenia oprogramowania i utraty kodu, oferując przejrzystą historię commitów, którą można cofnąć — ma to kluczowe znaczenie dla bezpiecznego zarządzania iteracyjnymi zmianami i ich testowania w złożonych projektach programistycznych.

3. Projektowanie

W procesie kompilacji będę używać CLion firmy JetBrains do kompilowania i pisania kodu. To nowoczesne środowisko IDE dla C++, a wybór konkretnego narzędzia zależy ostatecznie od preferencji, ponieważ pozostajemy przy wersji C++14.

Używam również GitHuba do pracy z repozytoriami, aby ćwiczyć bezpieczne przechowywanie danych.

Doxygen służy do podstawowego dokumentowania funkcji, metod i tym podobnych elementów w naszej bazie kodu. To sposób na ćwiczenie automatycznego dokumentowania kodu, ponieważ w większych projektach ręczne tworzenie dokumentacji może być niemal niemożliwe.

4. Implementacja

```
#include <iostream>
#include "factory_system.h"
#include "iterator_system.h"
#include "twosidelist.h"
using namespace std;

int main() {

    DoubleLinkedList list = factory_system::createSampleList();

    cout << "Display entire list: ";
    list.DLL_ForwardDisplay();

    cout << "Testing forward iteration on the sample list: ";
    iterator_system::iterateforward(list);

    cout << "Testing removing Index on the sample list: ";
    list.DLL_RemoveIDX(2);

    cout << "Display entire list [Post-Remove]: ";
    list.DLL_ForwardDisplay();

    cout << "Display Index 1 Slot: " << list.DLL_GetIDX(1) << endl;
    cout << "From opposite Index 1 Slot: " << list.DLL_GetEnd(1) <<
    endl;

    cout << "Next Node's Value: "; list.DLL_Next(); cout << endl;
    cout << "Previous Node's Value: ";  list.DLL_Prev(); cout <<
   endl;

    const int extra[] = {7, 8, 9};
    DoubleLinkedList fromArray = factory_system::createFromArray(
   extra, 3);
    cout << "Testing system from an array: ";
    iterator_system::iterateforward(fromArray);

    return 0;
}
```

Listing 1. Przykładowy kod 001

Ten kod jest przykładem funkcji main dla projektu. Pokazuje iteracje do przodu i do tyłu, pobieranie elementu od indeksu, wyświetlanie elementów od indeksu i

od końca, wyświetlanie następnego i poprzedniego węzła, a także pobieranie tablicy i konwertowanie jej do zaimplementowanego przez nas systemu węzłów, czyli list dwukierunkowych.

5. Wnioski

Implementacja pozwala nam przechowywać podstawowe dane na stercie za pomocą metody wytwórczej i iteracji. Metoda wytwórcza jest w tym przykładzie zbędna, ponieważ skala jest zdecydowanie zbyt mała, by zapewnić oszczędność pamięci, a iteracja opiera się na tym samym przykładzie. Zrobiono to, aby przygotować nas do przyszłych, bardziej złożonych projektów, ale skomplikowało to tworzenie projektu. W istocie nie jest od razu intuicyjne, jak działają oba modele/przykłady, nawet po intensywnej nauce.

Główne wnioski z tego doświadczenia dotyczą nauki obsługi gałęzi, wysyłania zmian i żądań ściągnięcia, zdalnych repozytoriów, żądań ściągnięcia, scalania, konfliktów i tym podobnych w GitHubie. Pokazało ono również, jak Doxygen służy do dokumentowania kodu.

Spis rysunków

1.1. Przykład jak lista działa 3

Spis tabel

Spis listingów

1. Przykładowy kod 001 7